

EVNet Sentinel

Leakage-Corrected Intrusion Detection on CICEVSE2024

EVNet Sentinel Team

September 29, 2026

Abstract

We reproduce the online intrusion-detection method of Makhmudov et al. (2025) on the CICEVSE2024 electric-vehicle charging dataset and find that its reported multiclass accuracy is an artifact of *capture-session timestamp leakage*. A decision tree given the single column `bidirectional_first_seen_ms` and nothing else recovers the 15-class label at 0.9976, exceeding the published 0.9840. The reference preprocessing compounds the problem by discarding `dst_port` — the feature that legitimately identifies a port scan — while retaining the timestamp that identifies it only by recording time. Removing both fingerprints collapses 1,724,504 reconnaissance flows to 6,821.

With the leak removed, flow-level features separate volumetric denial-of-service families near-perfectly (mean $F_1 = 1.000$, zero variance across seeds) but cannot distinguish `nmap` reconnaissance modes (mean $F_1 = 0.183$), and the two groups carry comparable support — so the gap is a property of the data rather than of model capacity. Separately, prequential accuracy in capture order is inflated a further 44 points by label autocorrelation, and 42 of 43 detected concept-drift events coincide with the seams where packet captures were concatenated.

1 Dataset and Pipeline

CICEVSE2024 is access-restricted and carries no public version string, so the capture set is pinned by fingerprint rather than by version.

Table 1: Dataset provenance.

Property	Value
Source	CICEVSE2024, Canadian Institute for Cybersecurity (network traffic subset)
Capture files	59
Raw flows	2,744,700
Classes	15 (14 attack types + benign)
Aggregate SHA-256	be68ff18bbb86f217c7c0852c988f1fa4503b79bbda11c89960cf2700b63ea07

Reproduce: `python3 src/reproduction/dataset_manifest.py -raw-dir data/raw`

Reproduce: `make data-process`

Labels are derived from the capture filename inside `preprocess.py`, porting the reference implementation’s 19-alias map. An earlier revision of this project depended on a pre-labelled private archive and could not rebuild the dataset from the public download; that step is now in version control.

Table 2: Preprocessing stages, `-feature-set` extended.

Stage	Rows	Columns
59 capture CSVs concatenated, labelled from filename	2,744,700	86 + label
After identifier, sparse, timestamp and <code>src_port</code> drops	2,744,700	65
After deduplication (<code>-dedup global</code>)	1,198,151	65
Feature matrix (metadata and label separated)	1,198,151	60
Train / validation / test (70/15/15)	838,705 / 179,723 / 179,723	60

2 Finding 1: The label is recoverable from a single timestamp column

The reference preprocessing retains six absolute epoch-millisecond columns: `bidirectional_first_seen_ms`, `bidirectional_last_seen_ms`, `src2dst_first_seen_ms`, `src2dst_last_seen_ms`, `dst2src_first_seen_ms` and `dst2src_last_seen_ms`.

Each attack in CICEVSE2024 was captured in its own packet capture at its own wall-clock time, so these columns constitute a capture-session identifier rather than a property of the traffic. None of the three drop stages in the reference notebook removes them.

Table 3: What a single feature recovers on its own. Decision tree, `max_depth=25`, 15-class target, 3-fold cross-validation.

Feature	Nature	Accuracy
<code>bidirectional_first_seen_ms</code>	capture fingerprint	0.9976
<code>src_port</code>	partial capture fingerprint	0.4028
<code>dst_port</code>	legitimate signal	0.2917
<i>Published multiclass accuracy (Makhmudov et al.)</i>		<i>0.9840</i>

A single-column decision stump exceeds the published result

Replaying the reference preprocessing verbatim yields 1,277,520 rows against its reported “~1.2M”, confirming the reconstruction is faithful. On that feature set, a tree using `bidirectional_first_seen_ms` alone scores 0.9976 — higher than the 0.9840 the paper reports for its full model. In our own pre-correction pipeline, 39.8% of the Random Forest’s total feature importance sat on those six columns.

Reproduce: `python3 src/reproduction/replay_reference_preprocessing.py -audit`

3 Finding 2: Reconnaissance rests on two columns

Reconnaissance flows are distinguished almost entirely by `dst_port` and the capture timestamp. Holding the reference feature set fixed and toggling only those two, then deduplicating as the reference does:

Reproduce: `python3 src/reproduction/ablate_fingerprints.py`

The emphasised row is the published preprocessing. Each column is independently near-sufficient, so neither alone explains the collapse — but the two are not equivalent in kind:

Table 4: Ablation over the two fingerprints. Flood classes are unaffected throughout (1,001,106 → 970,079, 96.9% retained).

Timestamps	dst_port	Total rows	Recon. flows	% of max
kept	kept	2,728,799	1,724,504	100.0 %
kept	dropped	1,277,520	273,932	15.9 %
dropped	kept	1,198,143	194,737	11.3 %
dropped	dropped	977,862	6,821	0.4 %

- `dst_port` is a **legitimate** discriminator. A port scan is *defined* by sweeping destination ports; that is the behaviour, not an artifact.
- The capture timestamp is **not**. It identifies which recording the flow came from.

The reference pipeline discards the real signal and keeps the artifact

By dropping `dst_port` while retaining the timestamps, the reference preprocessing removes the feature that legitimately identifies reconnaissance and keeps the one that identifies it only by accident of recording time. Its 273,932 reconnaissance flows are therefore separable by when they were captured rather than by what they did — which is precisely why a stump on `bidirectional_first_seen_ms` reaches 0.9976.

Correction to an earlier revision

An earlier version of this document attributed the reconnaissance collapse to the timestamps alone. The ablation above shows that dropping `dst_port` accounts for the larger share. The finding stands; the mechanism as previously stated did not.

4 Finding 3: Stream order, and what the drift detector is detecting

4.1 Label autocorrelation

Evaluated prequentially on an identical 25,000-instance slice with leak-free features, varying only the order in which the stream arrives:

Table 5: Stream ordering, controlled. `file-then-shuffle` uses the same instances and the same class distribution as `file`, differing only in order.

Order	Accuracy	Weighted F_1	Drift events
Capture order (<code>file</code>)	0.9976	0.9975	3
Shuffled, controlled (<code>file-then-shuffle</code>)	0.5508	0.5318	1
Shuffled, whole stream	0.5029	0.4909	1

In capture order, consecutive flows share a label, so a prequential learner scores near-perfectly by predicting “the same as recently”. This is an inflation mechanism entirely separate from the timestamp leak of Finding 1.

4.2 Drift events are capture-file seams

CICEVSE2024 is 59 packet captures concatenated end to end, and each join is an abrupt distribution change by construction. Running ADWIN prequentially over the full 1,921,182-instance stream in capture order:

Table 6: Alignment of detected drift events with capture-file boundaries, against a null model of 2,000 random placements.

Quantity	Value
Drift events detected	43
Events within 100 instances of a capture boundary	42
Median distance to the nearest boundary	22 instances
Expected under random placement	2.2 ± 1.5
Empirical significance	$p < 0.0005$

The detector is finding the seams, not the traffic

42 of 43 drift events land within 100 instances of a point where one recording was stitched to the next, against 2.2 ± 1.5 expected by chance. These events are not evidence that charging-station traffic evolves; they mark dataset assembly. This matters because the adaptivity of the online method is justified in the original work by the presence of concept drift.

5 Finding 4: `src_port` is a residual fingerprint

Ephemeral source ports are allocated near-sequentially by the operating system, so within a single capture they occupy a contiguous band — a lower-resolution version of the timestamp fingerprint.

Table 7: Controlled measurement: identical rows, identical split, column removed at fit time so deduplication cannot confound the comparison.

Model	With <code>src_port</code>	Without	Δ
Random Forest (depth 15, 100 trees)	0.5894	0.5138	-0.0756
Decision Tree (depth 20)	0.6834	0.5135	-0.1699

Reproduce: `python3 src/reproduction/srcport_control.py`

The deeper model loses more than twice as much, which is the signature of a memorable fingerprint rather than a genuine feature: additional depth buys finer `src_port` bands and nothing else. It also resolves an anomaly in the first round of corrected results, where an unrestricted Decision Tree appeared to outperform the ensemble. Once `src_port` is removed the two converge (0.5138 versus 0.5135), so that entire advantage was memorisation.

`dst_port` is retained: unlike `src_port` it is the behaviour that defines a port scan.

6 Corrected Results

Macro- F_1 is the headline, not accuracy

With ICMP_Fragmentation at 28 flows and Benign at 81, accuracy tracks the volumetric floods and compresses genuinely different models into the same number. The corrected Random Forest scores 0.8679 accuracy against 0.5582 macro- F_1 on identical predictions.

Table 8: Multiclass results, mean $\pm$ standard deviation over seeds 42 / 1337 / 2024. Each seed re-runs the train/test split and model initialisation.

Model	Macro F_1	Weighted F_1	Accuracy	Fit (s)
Random Forest	0.5582 $\pm$ 0.0080	0.8654	0.8679	8.6
Decision Tree	0.5568 $\pm$ 0.0095	0.8677	0.8681	3.5
Logistic Regression	0.4355 $\pm$ 0.0172	0.8529	0.8626	71.7
SVM (SGD, hinge)	0.4110 $\pm$ 0.0165	0.8489	0.8623	36.3

Reproduce: `python3 src/evaluation/run_experiments.py -raw-dir data/raw -feature-set extended`
Random Forest and Decision Tree overlap at one standard deviation, independently confirming Finding 4: their earlier apparent gap was `src_port` memorisation, not modelling.

6.1 The central result, per class

Table 9: Per-class F_1 , mean over three seeds. “Support” is the total flow count across all partitions after deduplication.

Class	Group	Support	RF	DT	LogReg	SVM
PSHACK Flood	Vol.	195,952	1.000	1.000	1.000	1.000
SynonymousIP Flood	Vol.	256,739	1.000	1.000	0.991	0.992
TCP Flood	Vol.	256,297	1.000	1.000	1.000	1.000
SYN Flood	Vol.	259,484	1.000	1.000	0.991	0.992
UDP Flood	Vol.	32,072	0.999	0.999	1.000	1.000
TCP Port Scan	Rec.	35,123	0.265	0.202	0.278	0.100
OS Fingerprinting	Rec.	27,702	0.223	0.227	0.039	0.022
Vulnerability Scan	Rec.	39,002	0.204	0.230	0.215	0.201
Service Version Det.	Rec.	30,338	0.156	0.174	0.076	0.027
Aggressive Scan	Rec.	27,815	0.145	0.179	0.081	0.041
SYN Stealth Scan	Rec.	34,765	0.105	0.175	0.042	0.183
Benign	Oth.	81	0.986	0.941	0.225	0.192
ICMP Flood	Oth.	32	0.660	0.638	0.216	0.178
ICMP Fragmentation	Oth.	28	0.439	0.422	0.095	0.094
Slowloris Scan	Oth.	2,721	0.191	0.166	0.283	0.145

The gap is a property of the data, not of model capacity

Volumetric floods reach mean $F_1 = 1.000$ with zero variance across seeds; reconnaissance sits at 0.183. The obvious objection — that reconnaissance simply has fewer examples — does not

hold: *UDP_Flood* has 32,072 flows and scores 0.999, while *TCP_Port_Scan* has 35,123 flows and scores 0.265. Comparable support, opposite outcomes.

All six reconnaissance classes are *nmap* modes, and *-A* is literally *-sV -O -sC -traceroute*, a superset of two of the others. At flow level they emit near-identical single-probe flows and should be confusable. This establishes a realistic ceiling for flow-based EVCS intrusion detection rather than a failure of the models.

The four classes in the “Other” group are excluded from both group means and reported separately: three have fewer than 100 flows in the entire dataset, and Slowloris is a low-rate attack whose flows resemble neither group.

6.2 Effect of removing the leak

Table 10: Macro- F_1 before and after removing the six timestamp columns, like-for-like models.

Model	Leaked	Corrected	Δ
Random Forest	0.9547	0.5894	− 0.3653
Logistic Regression	0.4622	0.3308	−0.1314
SVM (SGD, hinge)	0.2829	0.3084	+0.0255

The linear models barely move, because a hyperplane cannot exploit a timestamp axis the way a tree can. That — not non-linearity — is what the original 0.9994-versus-0.3665 gap was measuring.

7 Per-Attack Detection and Response

The results above answer “how good is each model”. An operator needs a different answer: *for this attack, does the system catch it, and which model should I trust to name it?* Every attack type was therefore fed to every model in 30 random batches of 50 unseen flows, repeated for each of the 3 independently trained model sets, and each verdict was scored at three levels:

- **Detected** — flagged as malicious at all, rather than called benign;
- **Category** — placed in the right response category, denial of service or reconnaissance;
- **Exact** — named as the exact attack class.

The levels matter because they fail differently. A port scan reported as an aggressive scan is still a scan, and the correct response is the same; a port scan passed as benign is a genuine miss.

Reproduce: `make attack-test`

The system does not fail silently

Across every mixed campaign, every model flagged at least 99.9% of attack flows as malicious. No attack type with adequate test data is passed as normal traffic by the best model for it. Where the system is weak, it is weak at naming, not at noticing.

Table 11: Exact / category rate per attack and model, with the resulting verdict: ID names the attack (≥ 0.90 exact), CAT right category only, DET flagged but wrong category, MISS passed as benign. For benign traffic the failure is a false alarm: OK or FA. *Fewer than 100 unseen flows; resampled, indicative only.

Attack	RF	DT	LR	SVM	Best
PSHACK Flood	1.00 / 1.00 ID	1.00 / 1.00 ID	1.00 / 1.00 ID	1.00 / 1.00 ID	all tie
SYN Flood	1.00 / 1.00 ID	1.00 / 1.00 ID	0.98 / 1.00 ID	0.99 / 1.00 ID	RF/DT
Slowloris Scan	0.82 / 0.82 DET	0.70 / 0.70 DET	0.52 / 0.64 DET	0.70 / 0.79 DET	RF
SynonymousIP Flood	1.00 / 1.00 ID	1.00 / 1.00 ID	1.00 / 1.00 ID	1.00 / 1.00 ID	all tie
TCP Flood	1.00 / 1.00 ID	1.00 / 1.00 ID	1.00 / 1.00 ID	1.00 / 1.00 ID	all tie
UDP Flood	1.00 / 1.00 ID	1.00 / 1.00 ID	1.00 / 1.00 ID	1.00 / 1.00 ID	all tie
ICMP Flood*	0.74 / 0.74 DET	0.73 / 0.73 DET	0.49 / 0.56 MISS	0.44 / 0.51 MISS	RF
ICMP Fragmentation*	0.49 / 0.81 DET	0.40 / 0.81 DET	0.58 / 0.81 MISS	0.46 / 0.82 MISS	LR
Aggressive Scan	0.12 / 0.90 DET	0.15 / 0.90 DET	0.09 / 0.94 CAT	0.02 / 0.85 DET	DT
OS Fingerprinting	0.41 / 0.88 DET	0.41 / 0.89 DET	0.02 / 0.95 CAT	0.01 / 0.86 DET	RF/DT
SYN Stealth Scan	0.06 / 0.91 CAT	0.14 / 0.91 CAT	0.03 / 0.97 CAT	0.43 / 0.90 DET	SVM
Service Version Detection	0.10 / 0.89 DET	0.13 / 0.90 DET	0.04 / 0.96 CAT	0.01 / 0.87 DET	DT
TCP Port Scan	0.32 / 0.91 CAT	0.18 / 0.91 CAT	0.65 / 0.97 CAT	0.25 / 0.89 DET	LR
Vulnerability Scan	0.13 / 0.92 CAT	0.17 / 0.92 CAT	0.20 / 0.97 CAT	0.26 / 0.91 CAT	SVM
Benign*	0.98 / 0.98 OK	0.88 / 0.88 FA	0.54 / 0.54 FA	0.41 / 0.41 FA	RF

Table 12: Mixed campaigns: 90 trials of four simultaneous attacks, 25 flows each, shuffled together.

Model	Detected	Category	Exact
Random Forest	1.000	0.932	0.564
Decision Tree	1.000	0.923	0.560
Logistic Regression	0.999	0.949	0.529
SVM	1.000	0.920	0.560

7.1 Which model for which attack

- **Volumetric floods** (SYN, TCP, UDP, PSH-ACK, synonymous-IP) are named exactly by all four models; they tie.
- **Reconnaissance**: no model names scans reliably, but Logistic Regression is best at recognising that traffic *is* a scan — the right category on every scan type. Different models are best at naming different scans, all at low absolute rates.
- **Slowloris** is flagged but often misfiled: its long-lived, near-idle connections resemble slow scans. The same effect runs the other way — Random Forest loses 8–11 % of every scan type to Slowloris, which accounts for almost all cross-category error.
- **False alarms** on benign traffic differ sharply: Random Forest 2 %, Decision Tree 12 %, Logistic Regression 46 %, SVM 59 % — on only 12 unseen benign flows per model set, so indicative.

Recommendation: Random Forest as the primary detector

Among the models that name attacks as well as the best one, Random Forest raises the fewest false alarms. Logistic Regression is a useful second opinion for the narrower question of whether traffic is a scan. Where the primary detector cannot reliably name an attack, the dashboard and

simulation console respond at the category level, and report the category’s severity range rather than a severity derived from an untrusted name.

7.2 Response guidance

For each attack type the system states what it is, why it matters for a charging network, and what the station owner, CSMS operator and network analyst should do — held in `src/evaluation/response_playbook.json`, shown on the dashboard and in the simulation console alongside every detection. These are standard defensive measures for each attack mechanism, adapted to OCPP and ISO 15118; they are operator guidance and were not themselves evaluated here.

8 Reproducibility

Table 13: Environment in which the reported numbers were produced.

Component	Value
CPU	Apple M5 (10 logical cores)
Memory	16 GB
Python	3.13.12
NumPy	2.4.4
pandas	3.0.2
scikit-learn	1.8.0
River	0.26.1

`requirements.txt` pins exact versions. River’s API changed materially around 0.21, and scikit-learn’s `class_weight` handling and tree-splitting behaviour have shifted across minor releases, so unpinned versions make these numbers unreproducible.

Model artifacts are deliberately not version-controlled: `src/api/main.py` loads every `.pkl` in `saved_models/` into its serving registry, so a stale committed model is served silently as though it were current. Until this was corrected, the models in that directory were trained on leaked features.

9 Limitations and Future Work

9.1 Acknowledged limitations

- **The benign class is effectively absent.** The network-traffic subset contains 81 benign flows in total. A binary attack-versus-benign task is therefore answered by predicting “attack” unconditionally, and was removed from this work. Reported binary accuracies on this dataset, including the original 0.9913, should be read with that in mind.
- **Grouped and temporal splits are implemented but not used for headline numbers.** `preprocess.py` supports holding out whole captures and splitting by time; the reported results use a random split for comparability with prior work.
- **Host-events and power-consumption domains are unexplored.** CICEVSE2024 also provides hardware performance counters and power traces, which may separate the reconnaissance classes that flow-level features cannot.

9.2 Planned: a sequence-model comparator

A recurrent comparator (LSTM or GRU) is a natural next step, and would test whether the reconnaissance ceiling established here is a limit of *flow-level features* or of *non-sequential models*. Two design constraints follow directly from the findings above and must be respected, or the comparator will reproduce the artifacts this work set out to remove.

A sequence model over an ordered stream will re-introduce Finding 3

*Training an LSTM on flows in capture order hands it exactly the label autocorrelation that inflates prequential accuracy by 44 points. Consecutive flows share a label, so a model with memory can reach near-perfect accuracy by learning “predict what came recently” — and will appear to beat every static model here. Any sequence model must be evaluated against the *file-then-shuffle* control, and its reported gain measured relative to a trivial “repeat the last label” baseline.*

The defensible formulations are:

1. **Intra-flow packet sequences.** Model the packet-level sequence *within* a single flow rather than a sequence of flows. This is the formulation that could genuinely beat the ceiling, because it accesses structure that NFStream’s aggregation discards — the ordering of packet sizes and inter-arrival times that might separate one `nmap` mode from another. It requires re-extracting from the `.pcap` files rather than using the provided flow CSVs.
2. **Grouped-split sequences of flows.** Keep flow sequences, but hold out whole captures (`-split grouped -dedup per-capture`) so the model cannot exploit position within a capture it has already seen. Report against the autocorrelation baseline.

Formulation 1 is the one worth pursuing: it asks a question the current results cannot answer. Formulation 2 mainly measures how well a model memorises capture structure, which this work has already shown to be the dominant confound in this dataset.

Artifact Index

Script	Produces
<code>src/data_prep/preprocess.py</code>	The corrected dataset; feature-set, split, dedup and scaling options
<code>src/evaluation/run_experiments.py</code>	Every headline number, multi-seed
<code>src/reproduction/replay_reference_preprocessing.py</code>	Faithful replay of the reference pipeline, with leak audit
<code>src/reproduction/ablate_fingerprints.py</code>	The Finding 2 ablation
<code>src/reproduction/srcport_control.py</code>	The Finding 4 controlled measurement
<code>src/reproduction/dataset_manifest.py</code>	Dataset fingerprint
<code>src/models/arfadwin/train_arfadwin.py</code>	Prequential ARF + ADWIN, drift log with capture provenance
<code>src/evaluation/attack_test_harness.py</code>	Per-attack three-level tests, mixed campaigns, regression gates
<code>src/evaluation/response_playbook.json</code>	Response guidance per attack and role

Reference implementation: https://github.com/TATU-hacker/Intrusion_Detection_on_Electric_Vehicle_Charging_Systems

Makhmudov, F., Kilichev, D., Giyosov, U., Akhmedov, F. (2025). *Online Machine Learning for Intrusion Detection in Electric Vehicle Charging Systems*. Mathematics 13(5), 712. DOI: 10.3390/math13050712